Figure 9.18 Design option 3 for the Scripts data product

Options 2 and 3 are similar to the CQRS pattern (for more details, see "CQRS" by Martin Fowler, https://martinfowler.com/bliki/CQRS.html).

NOTE When applying the CQRS pattern to a service, we distinguish two conceptual models of our data: a command model optimized for updates and a query model optimized for reads. This pattern is mainly used in microservice architecture. By applying this pattern, multiple microservices can serve as different command models. But when there is a need to expose data in an aggregated way, we create a new service that consumes events from command services and makes an aggregated view. Using events and async communication is not a mandatory part of this pattern, so you can implement it using the same database for the command and query model, but with the model exposed through an API that is different.

In option 2, we are reusing the same database. To optimize query performance, we could use a database materialized view. Because we're using the same database, this solution is tightly coupled, but we gain the independently deployable Scripts data product service, which increases the complexity of the infrastructure.

In option 3, we have loosely coupled services that follow a "database per service pattern." In this design, we trade off simplicity for loose coupling.

After analyzing all options as a team, we decide to follow the KISS principle in this specific case and choose option 1.

9.3.4 Exposing data for stream processing and batch processing

In the following example, we will move away from Hitchcock Movie Maker and jump into a fancier and more advanced part of Messflix: its streaming platform. We want to detect fraud based on data coming from the streaming platform. In figure 9.19, you can see planned data products.